

Integrating a Traditional Chatbot with Azure AI Language Service

Introduction

Chatbots are widely used to communicate with users and answer common questions. Traditional chatbots work using predefined rules and conditions, while modern chatbots can also use Artificial Intelligence to understand user input better.

In this project, I integrated a traditional chatbot with Azure AI Language Service. The chatbot was developed using Microsoft Bot Framework and Python. The chatbot can answer simple commands and also use Azure AI to analyze the sentiment of a user's message. This project helped me understand how a traditional chatbot can be improved by connecting it to a cloud-based AI service.

The main goal of this project was to combine rule-based conversation with Artificial Intelligence while keeping the chatbot simple and easy to understand.

Project Objective

The objective of this project was to build a traditional chatbot and connect it to Azure AI Language Service.

The chatbot should:

- Welcome users.
- Display available commands.
- Accept user feedback.
- Send the feedback to Azure AI.
- Display the sentiment result.
- Handle invalid input properly.

- End the conversation politely.

The project also helped me understand how cloud AI services communicate with Python applications.

Chatbot Development Lifecycle

Planning

Before writing the code, I planned the chatbot functionality. Instead of creating another general chatbot, I selected a campus feedback assistant. Students can enter their experience, and the chatbot analyzes whether the feedback is positive, negative, neutral, or mixed.

This idea combines a traditional chatbot with Artificial Intelligence and demonstrates how AI services can improve user interaction.

Requirement Analysis

I identified the following requirements before development:

- Build a chatbot using Microsoft Bot Framework.
- Connect the chatbot to Azure AI Language Service.
- Analyze user feedback using sentiment analysis.
- Display the sentiment result.
- Provide a help menu.
- Handle incorrect input.
- Keep Azure credentials secure by using environment variables.

These requirements helped organize the project before coding.

Design

The chatbot uses two different approaches.

The first approach is a traditional rule-based chatbot. Commands such as **hello**, **help**, **about**, and **bye** are handled using predefined if-else conditions.

The second approach uses Azure AI Language Service. When the user enters feedback, the chatbot sends the text to Azure AI through an API request. Azure analyzes the message and returns the sentiment with confidence scores. The chatbot then displays the results to the user.

This design combines traditional programming with cloud-based Artificial Intelligence.

Development

The chatbot was developed using Python and Microsoft Bot Framework.

I first copied the Microsoft Echo Bot sample and confirmed that it worked correctly in Bot Framework Emulator. After verifying the sample, I modified the chatbot logic according to the project requirements.

Next, I created an Azure AI Language resource using the Azure for Students subscription. After the resource was created, I collected the endpoint and API key from Azure.

Instead of storing these values inside the source code, I used environment variables. This improves security because sensitive information is not stored inside the Python files.

Finally, I installed the Azure AI Text Analytics SDK and integrated sentiment analysis into the chatbot.

Testing

After completing the development, I tested the chatbot using Bot Framework Emulator.

The following scenarios were tested:

- Greeting the chatbot.
- Viewing the help menu.
- Sending positive feedback.
- Sending negative feedback.
- Sending neutral feedback.
- Sending mixed feedback.
- Entering unsupported messages.
- Exiting the chatbot.

The chatbot successfully returned sentiment analysis results for the different test cases.

Maintenance

Although the chatbot is working successfully, many improvements can be made in future versions.

Future enhancements may include:

- Saving user feedback in a database.
- Generating ticket numbers.
- Displaying charts based on sentiment.
- Sending email notifications.
- Supporting multiple languages.

- Integrating Azure OpenAI for natural conversations.

These improvements can make the chatbot more useful in a real university environment.

Azure AI Integration

Azure AI Language Service was used to analyze user feedback.

The chatbot sends the user's message to Azure AI using the Azure Text Analytics SDK.

Azure processes the message and returns the sentiment category along with confidence scores.

The chatbot displays these results to the user in a simple format.

Using Azure AI made the chatbot much smarter because it can understand the emotional meaning of a sentence instead of simply matching keywords.

Implementation

The project contains the following important files:

- app.py
- config.py
- bots/echo_bot.py
- requirements.txt
- README.md

The chatbot starts normally using Microsoft Bot Framework.

When the user enters a predefined command, the chatbot responds using rule-based logic.

When the user submits feedback, Azure AI analyzes the sentence and returns the sentiment result.

This implementation demonstrates how traditional chatbot logic and cloud AI services can work together.

Testing Results

The chatbot successfully performed the expected functionality.

The following features were verified:

- Welcome message
- Help command
- Positive sentiment detection
- Negative sentiment detection
- Neutral sentiment detection
- Mixed sentiment detection
- Invalid input handling
- Exit command

The chatbot responded correctly during all planned testing scenarios.

Interesting and Exciting Parts

The most interesting part of this project was connecting the chatbot to Azure AI Language Service.

It was exciting to see that the chatbot could understand the feeling behind a sentence instead of simply checking predefined keywords.

Another interesting part was using Bot Framework Emulator to test the chatbot locally without deploying it to Azure.

I also learned how environment variables improve application security by protecting sensitive API keys.

Areas That Need Improvement

Although the chatbot works correctly, there are several areas that can be improved.

Currently, the chatbot only performs sentiment analysis. It does not remember previous conversations or store user feedback.

The chatbot also depends on an internet connection because Azure AI processes the requests online.

Another improvement would be supporting voice input and multiple languages.

Challenges

This project was more challenging than creating a simple chatbot.

Initially, I faced issues while creating the Azure AI Language resource because my Azure subscription allowed only certain deployment regions. After selecting a supported region, the resource was created successfully.

Another challenge was configuring the environment variables correctly. Since the chatbot reads the Azure endpoint and API key from the operating system, I had to verify that the variables were available before running the application.

Understanding how Azure AI communicates with the chatbot also required additional learning. After testing different scenarios, I understood the overall integration process.

These challenges improved my confidence in cloud-based AI development.

Lessons Learned

This project helped me learn many new concepts.

I learned the difference between traditional rule-based chatbots and AI-assisted chatbots.

I learned how Microsoft Bot Framework handles chatbot conversations.

I learned how Azure AI Language Service performs sentiment analysis.

I also learned how to securely manage API credentials using environment variables.

Overall, this project improved my understanding of chatbot development, cloud services, API integration, testing, and debugging.

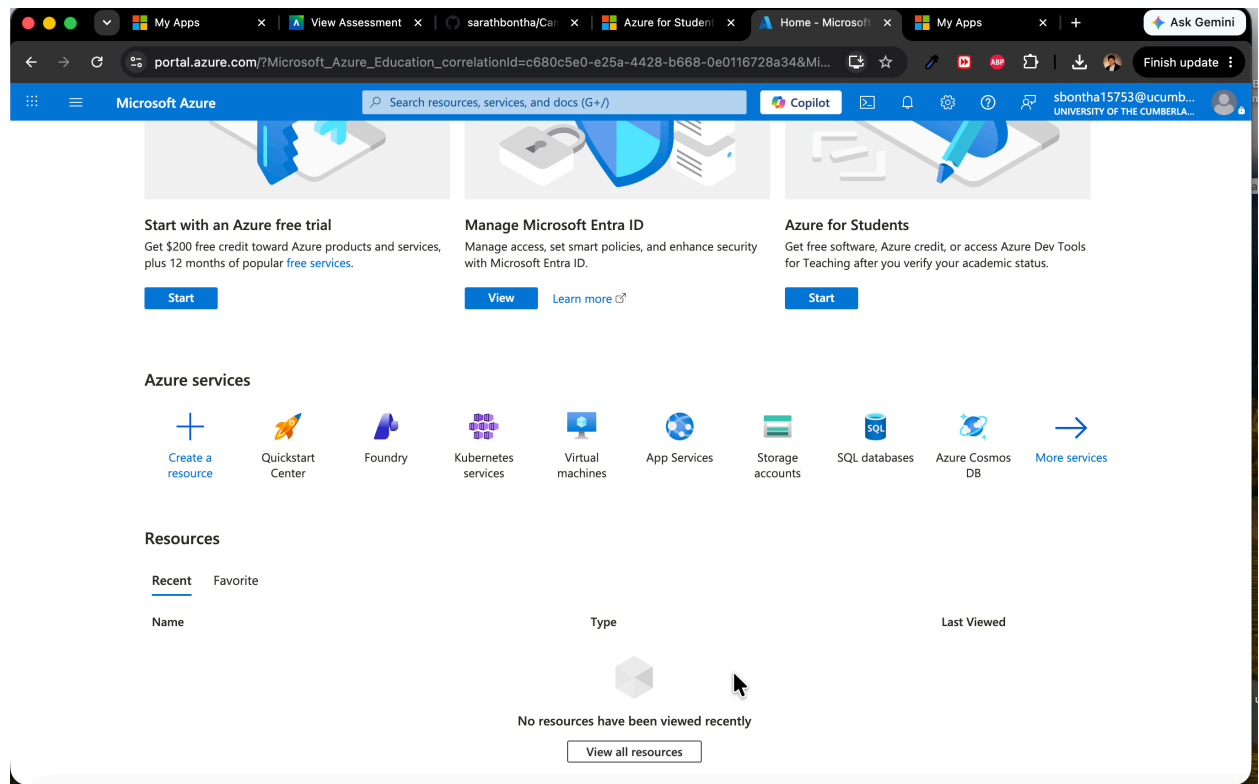
AI Use Disclosure

Artificial Intelligence was used to assist with understanding Microsoft Bot Framework, Azure AI Language integration, code explanations, report organization, and grammar improvements.

The chatbot configuration, Azure resource creation, coding, testing, screenshots, project execution, and final review were completed by the student.

Screenshots

Figure 1. Azure Portal Home



The figure shows the Azure Portal after successfully signing into the Azure for Students subscription.

Figure 2. Azure AI Language Resource

Create Language ...

BasicsNetworkIdentityTagsReview + create

Unlock insights from unstructured text using advanced natural language processing. Use sentiment analysis to find out what customers think of your brand. Find topic-relevant phrases using key phrase extraction and identify the language of the text with language detection. Detect and categorize entities in your text with named entity recognition.

[Learn more](#)

Project Details

Subscription * ⓘ

Azure for Students

Resource group * ⓘ

(New) CampusCareRG

Create new

ⓘ

Azure AI services resource creation requires subscription registration, we detected that your selected subscription did not register Cognitive services resource type before, we will help you to register Cognitive services resource type when you select a subscription in subscription dropdown. Click to learn more how to check registration state for your selected subscription.

Instance Details

Region ⓘ

(US) East US

Name * ⓘ

campuslanguage2026

Pricing tier * ⓘ

Free F0 (5K Transactions per 30 days)

[View full pricing details](#)

Responsible AI Notice

Previous

Next

Review + create

Create Language ...

- Basics
- Network
- Identity
- Tags
- Review + create

[View automation template](#)

Basics

Subscription	Azure for Students
Resource group	CampusCareRG
Region	South Central US
Name	campuslanguage2026
Pricing tier	Free F0 (5K Transactions per 30 days)

Network

Type	All networks, including the internet, can access this resource.
------	---

Identity

Identity type	SystemAssigned
---------------	----------------

portal.azure.com/view/Microsoft_Azure_Resources/DeploymentDetails.MenuView/-/overview?id/%2Fsubscriptions%2F5fa195a...

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

Incognito

Finish update

Home

TextAnalyticsCreate-20260719175022 | Overview

Deployment

Search

Delete Cancel Redeploy Download Refresh

Overview

- Inputs
- Outputs
- Template

Your deployment is complete

Deployment name : TextAnalyticsCreate-20260719175022 Start time : 7/19/2026, 5:55:30 PM
Subscription : Azure for Students Correlation ID : b8deb93b-154f-4f48-8b72-3791a5cd79f1
Resource group : CampusCareRG

> Deployment details

< Next steps

Go to resource

Cost management

Get notified to stay within your budget and prevent unexpected charges on your bill.
[Set up cost alerts >](#)

Microsoft Defender for Cloud

Secure your apps and infrastructure
[Go to Microsoft Defender for Cloud >](#)

Free Microsoft tutorials

[Start learning today >](#)

Work with an expert

Azure experts are service provider partners who can help manage your assets on Azure and be your first line of support.
[Find an Azure expert >](#)

Add or remove favorites by pressing Ctrl+Shift+F

portal.azure.com/#@cumber.onmicrosoft.com/resource/subscriptions/5fa195a4-2985-4a64-8dd2-dfc7e0c73c/resourceGrou...

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

Incognito

Finish update

Home > TextAnalyticsCreate-20260719175022 | Overview

CampusCareRG

Resource group

How to manage these changes more efficiently with deployment tools? Help me generate Terraform for this resource group configuration. +1

Search

Create Manage view Delete resource group Refresh Export to CSV Open query Assign tags Move Delete Group by none

Overview

- Activity log
- Access control (IAM)
- Tags
- Resource visualizer
- Events
- Settings
- Cost Management
- Monitoring
- Automation
- Help

Essentials

Resources Recommendations

Filter for any field... Type equals all Location equals all Add filter

Name ↑	Type	Location
campuslanguage2026	Language	South Central US

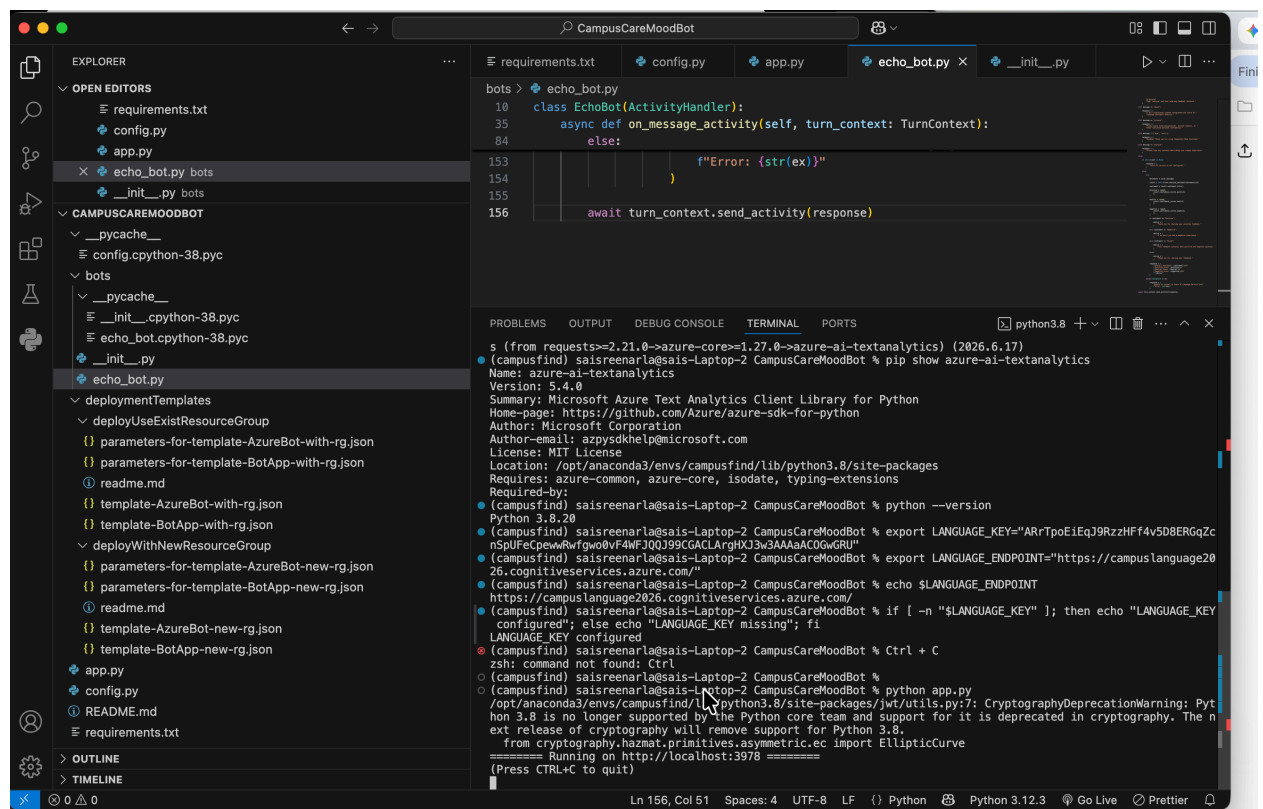
Showing 1 - 1 of 1. Display count: auto

Add or remove favorites by pressing Ctrl+Shift+F

[Give feedback](#)

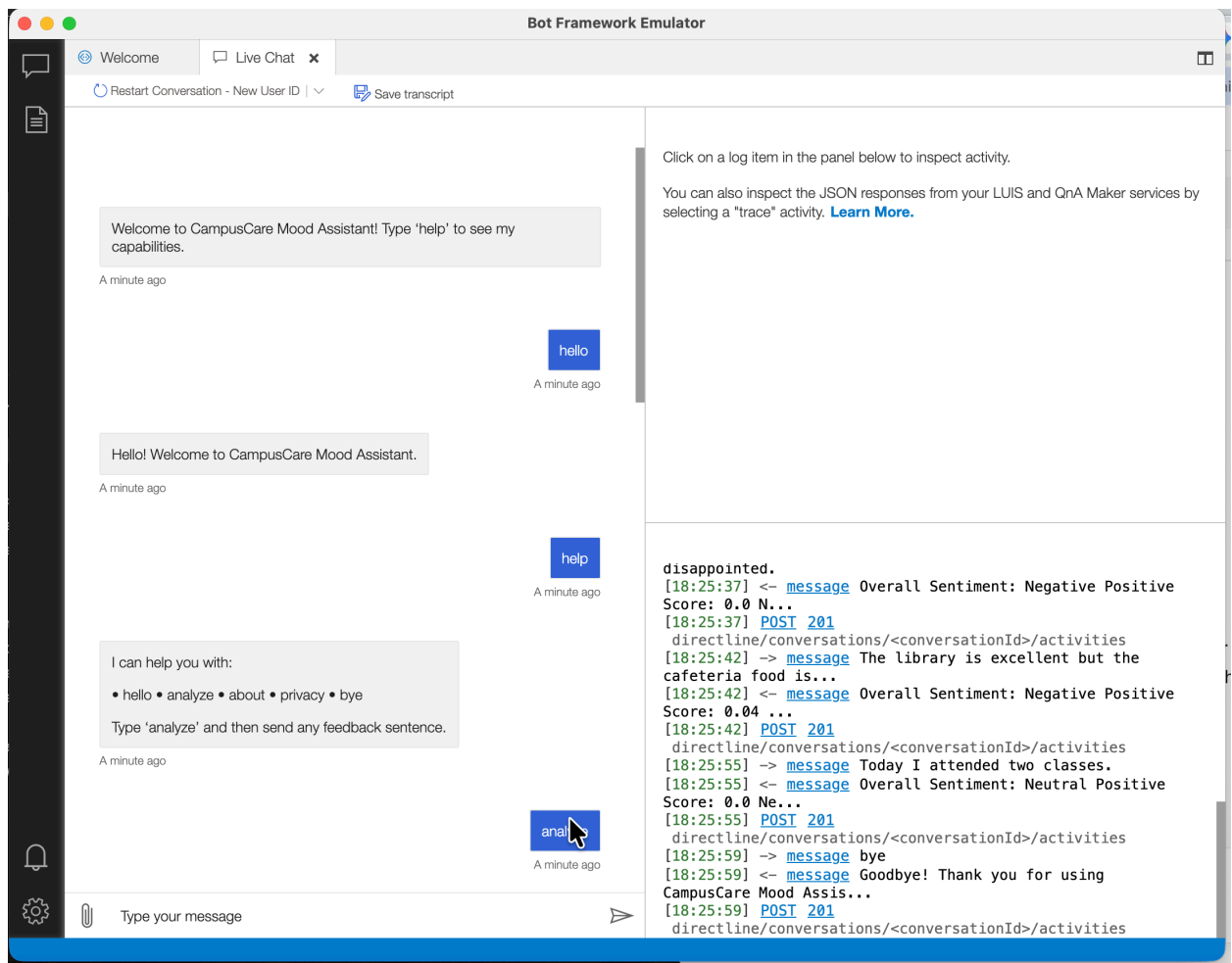
The figure shows the Azure AI Language resource created for this project.

Figure 3. Bot Running



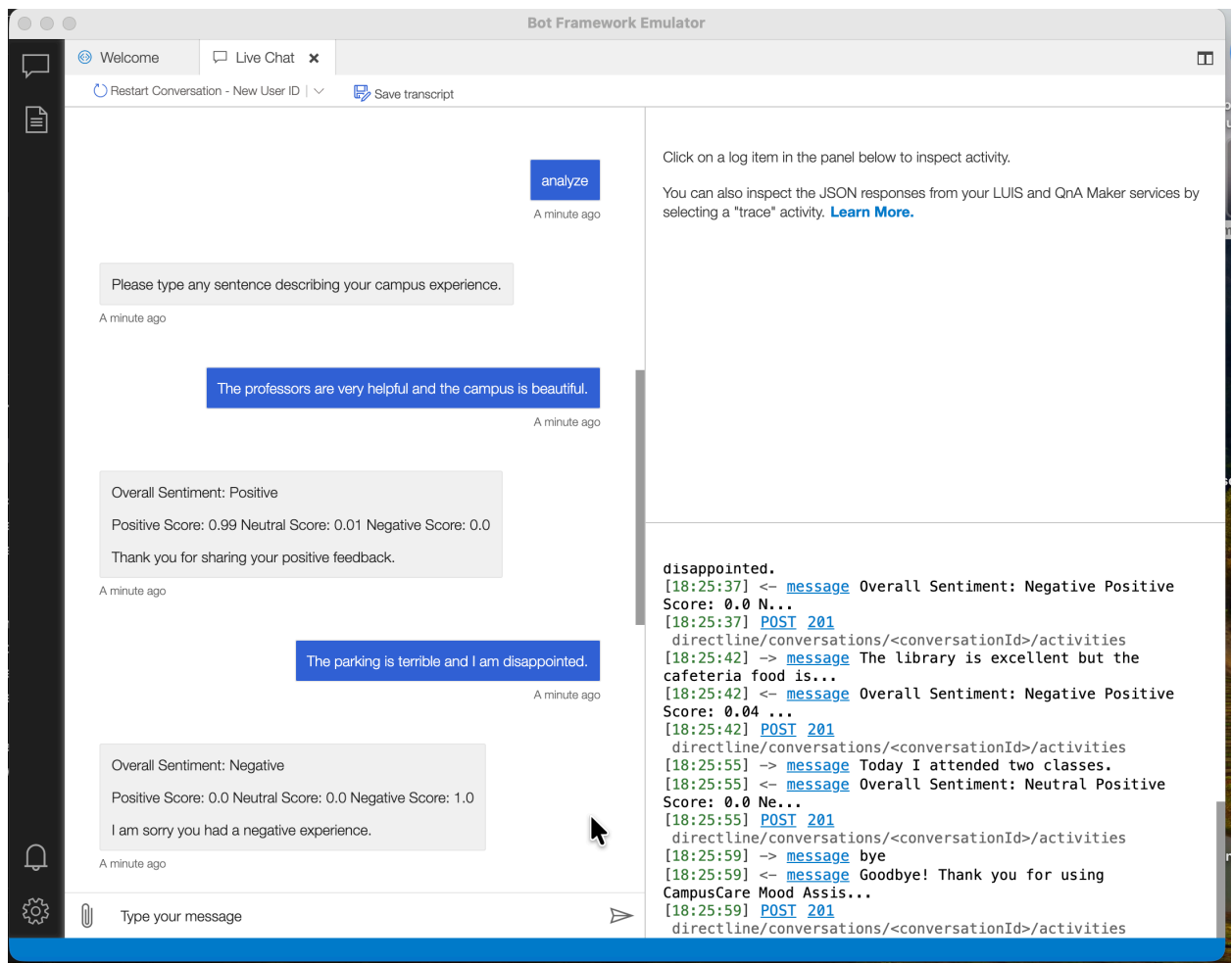
The figure shows the chatbot application running successfully on the local server.

Figure 4. Bot Framework Emulator



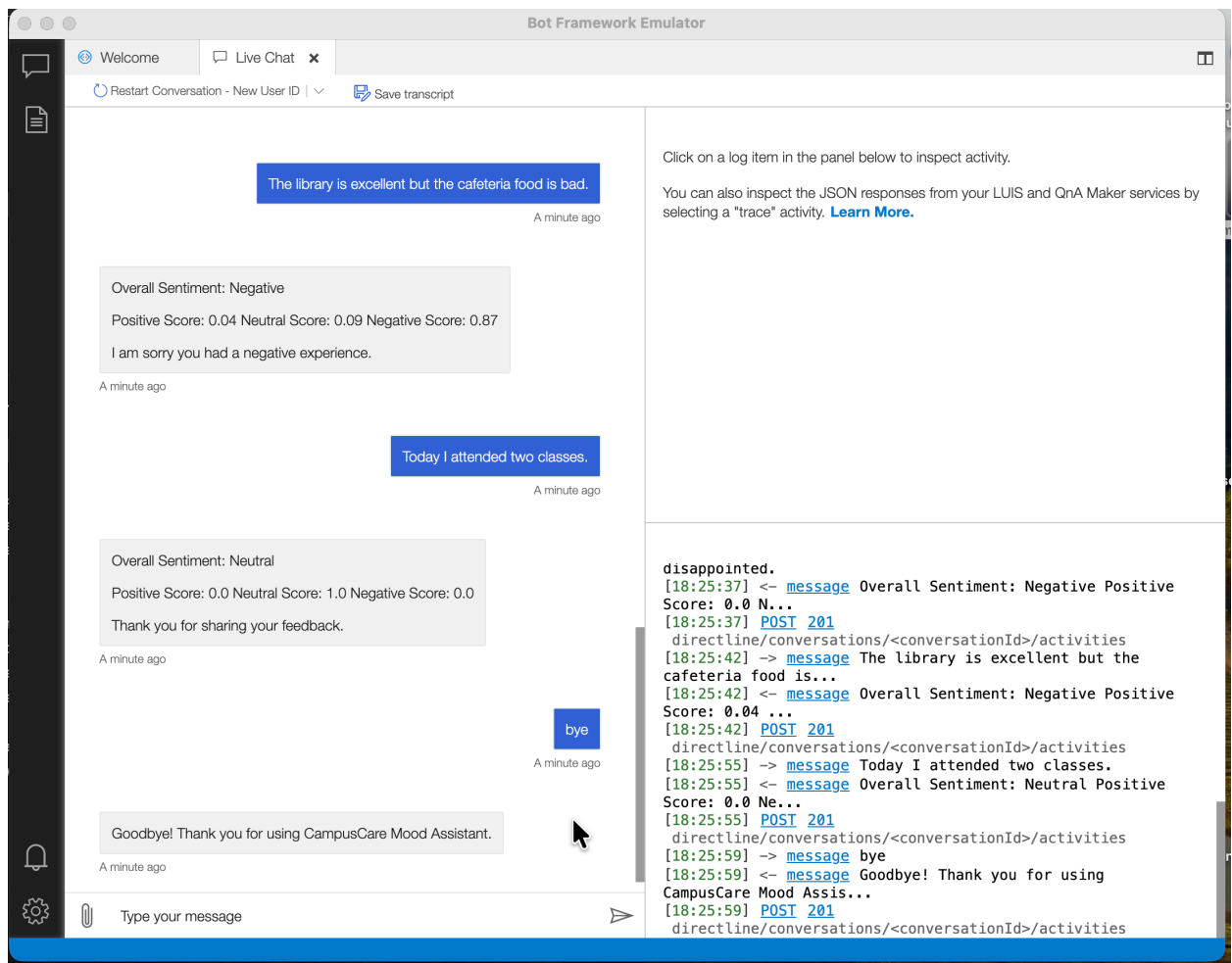
The figure shows the chatbot connected to Bot Framework Emulator.

Figure 5. Positive Sentiment and Negative Sentiment



The figure shows Azure AI identifying positive user feedback and negative user feedback.

Figure 6. Neutral Sentiment



The figure shows Azure AI identifying neutral feedback.

Conclusion

This project provided practical experience in integrating a traditional chatbot with a cloud-based Artificial Intelligence service.

The combination of Microsoft Bot Framework and Azure AI Language Service helped me understand how rule-based chatbots can become more intelligent by using cloud APIs.

The project also improved my knowledge of Azure services, Python programming, API integration, chatbot development, software testing, and secure application configuration.

Overall, this project was a valuable learning experience and provided a strong foundation for developing more advanced AI-powered chatbot applications in the future.

GitHub Repository: <https://github.com/sarathbontha/CampusCareMoodBot>

References

Anaconda. (2017). *Managing environments*.

<https://conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html>

Microsoft. (n.d.). *Azure AI Language documentation*.

<https://learn.microsoft.com/azure/ai-services/language-service/>

Microsoft. (n.d.). *Microsoft Bot Framework*. <https://dev.botframework.com/>

Python Software Foundation. (n.d.). *Python documentation*. <https://docs.python.org/3/>